ADVANCED AI SOCIETY · PROOF-OF-CONTROL INITIATIVE
Working draft v0.1 · August 2026 · for co-author review prior to submission

What Unlinkability Costs at Agent Action Rates: A Composed-Path Measurement and the Soundness of Pre-Computation

The obvious construction misses the budget, and the fix is a security question nobody has asked.

Jim Schwoebel[1,2]

[1] Quome [2] Advanced AI Society, `https://advancedaisociety.org`[1]

ABSTRACT

An agent presents an anonymous credential and a relying party verifies it, and the standard gives the two of them together one 15.00 ms action budget. A desk study checked them one at a time. BBS+ presentation, about 13.80 ms: under. Verification, about 4.20 ms: under. Two halves under budget, so the design fits — except that the action pays for both, and the sum is 18.00 ms, over by 3.00 ms. With cached pairing pre-computation the sum is 14.90 ms, under the limit by 100 µs, which this paper's tool reports as `MARGINAL` rather than a pass, because 100 µs is not headroom for the application, the network, or writing the evidence record. We built `occultation`, a Rust tool that assesses presentation and verification as one composed path rather than two separate ones, over real BBS+ signatures on BLS12-381 and a real Ed25519 baseline. Our hardware is much faster — the composed path costs 14× less here than the desk study's published figures imply, or 10× once the modelled escrow tag is excluded from both sides — and none of that touches the mistake, because the mistake is in the addition and not in the silicon. What the magnitude would hide is that the *shape* of the correction survives the hardware change while the shape of an unrelated optimization does not: cached pairings save 74 % of verification in the desk study's figures and 11 % of it in ours. Pre-computation is what makes the desk study's own published figures fit the budget at all, which converts a performance question into a security one: what happens when the pre-computed pool runs out under burst? We simulate it. The pool stalls; it does not reuse, and reuse is not configurable, because we show precisely what reuse costs — an observer of two presentations sharing one blinding factor recovers the signature scalar and every undisclosed attribute by subtraction, an extraction our test suite performs rather than merely asserts. Stalling is not free either: under a sustained 5000 s^{-1} burst against a ∼1429 s^{-1} refill rate, the worst response in a 2.0 s window is 82 591× an idle one, a timing side channel with a duration-invariant signature — overload factor 3.50×, growth per request 8.33× an idle response, opening at request 89. Separately, we ask how much of the unlinkability the hardware layer underneath hands straight back. Partitioning a fleet description — a synthetic, hand-authored fixture, not telemetry from any live fleet — 120 identical hosts, after six weeks of the kind of operational drift in patch level and microcode such a fleet would accumulate, fall into 5 distinguishable groups of 71, 34, 11, 3, 1. 1 host is alone on its configuration before any credential

[1] Prepared within the Proof-of-Control Initiative of the Advanced AI Society. Authorship is limited to contributors to this manuscript; additional co-authors will be added only upon explicit consent and substantive contribution.

is even presented. Effective set size 52.73 of 120; entropy 1.470 bits. Two of the five layers this paper's subject construction proposes — hardware attestation and threshold escrow — are implemented here only as cost-representative stubs with no security whatsoever, and every number that touches them says so in the sentence that states it, not in a footnote.

Artifact. `occultation` [1] is Apache-2.0. Every number in Section 5 is regenerated from the tool by `scripts/regen-results.sh` and read into this document from the generated files under `../results/`; continuous integration regenerates the host-independent half of that directory and fails on a diff, so a stale number cannot outlive a code change. The other half is wall-clock timing, differs by host, and is labelled as such everywhere it appears.

Contents

1 The Problem **4**

2 What Others Have Built **5**

3 The Construction We Measured **6**

4 Measuring the Composed Path **7**

5 What It Actually Costs **8**
5.1 Adding the two halves together 8
5.2 What the pool costs when it runs dry 10
5.3 How much of the fleet looks alike 10

6 Is the Pre-Computation Pool Sound? **11**

7 What We Still Don't Know **12**

8 Conclusion **13**

1 The Problem

An enterprise sends a software agent out to buy something from a vendor. The agent presents a credential, the vendor checks it, the order goes through. Nobody did anything wrong.

Now ask what the vendor learned.

It learned which company the agent belongs to, because the credential said so. It learned which employee set the agent going, because the evidence record the standard asks for carries an `initiating_user` field. It can count how many times that same agent came back this month, because every action is bound into a hash chain whose roots are anchored where anybody can read them. And it can line all of that up against every other agent from the same firm, because the same identifiers run through all of them. By the end of the quarter the vendor holds a running account of another company's internal operations, assembled as a by-product of checking purchase orders.

The vendor had one question: is this agent allowed to do this. None of the rest was needed to answer it.

That gap is open issue 2 in C5, the identity domain of the Proof-of-Control standard [2] — six requirements, the thinnest of the standard's six domains. Can an autonomous agent's actions be independently verifiable and attributable when it matters, and unlinkable across relying parties the rest of the time? A working-group member raised it in the plainest possible terms: enterprises need to deploy agents without disclosing customer PII or corporate identity to third-party vendors, and the best way to secure information is not to hand it out unnecessarily.

The tension is structural, not incidental. Every other part of the standard this domain sits inside pulls the other way, toward *more* identifiable evidence: an explicit `agent_id`, an `initiating_user`, a hash chain that links every action to every action before it, anchors published where anyone can see them. If you set out to design the worst possible object for unlinkability you would land somewhere near that: a permanent, ordered, cryptographically bound dossier. You do not fix it by leaving a field out of a record. You need a credential layer whose presentations do not link to each other, sitting underneath a log layer built, by design, to chain everything together.

The cryptography for that layer is not new. The anonymous-credential line runs back two decades, and Section 2 walks it. So why is nobody running it here? Because nobody had measured what it costs at agent action rates. One desk study estimated it: a concrete five-layer answer — hardware attestation, an anonymous credential, an evidence log, a threshold escrow for accountability, and a transport layer — priced from published benchmarks of other people's hardware rather than from anything it ran. We checked the arithmetic. It contains an error large enough to flip the study's own headline conclusion, and finding that error is most of what this paper is about.

Here is the error. The standard allows 15.00 ms per action. The desk study weighed presentation against that budget: about 13.80 ms, under. Then it weighed verification: about 4.20 ms, also under. Two halves under budget, so the design fits.

An agent that presents a credential and has it verified pays for both. 18.00 ms. That is over.

No cryptographic assumption in the desk study is wrong. The mistake is in the addition, and it survives being pointed out only because presentation and verification happen on different machines at different times, which makes it easy to grade them on separate report cards.

This paper reports what happens when that composed path, and the two security questions the desk study's own numbers raise but do not answer — whether the fix for the composition error is itself sound, and how large an anonymity set the hardware layer actually leaves standing — are measured rather than estimated.

One thing we are not claiming. Two of the five layers below are stubs with no security in them whatsoever. Our BBS+ is real cryptography that will not interoperate with anybody else's. The fleet whose anonymity we partition is a file we wrote by hand, not telemetry from a live one. Every figure in this paper sits inside those limits, and each limit is repeated in the sentence that states the number rather than parked in a footnote.

2 What Others Have Built

Every mechanism this design needs already exists and is trusted. Pointing them at a machine-speed, hash-chained, publicly anchored agent log is the part nobody has done. So every citation below does double duty: it shows the primitive is real, and it marks the distance between that primitive and this use of it.

Hardware attestation without a platform identifier. Direct Anonymous Attestation [3] lets a TPM prove that *some* valid TPM produced a statement without revealing which one. It is adopted in TPM 2.0 and standardized as ISO/IEC 20008 [4]. The elliptic-curve variant, ECDAA, is the practical form used in FIDO and is the closest existing answer to the hardware layer this construction needs. It is also the layer we do not implement: Section 3 explains why it is modelled rather than built.

Anonymous credentials. Camenisch–Lysyanskaya credentials [5], Idemix [6], and U-Prove [7] are the lineage this paper's credential layer descends from. BBS+ signatures over pairing-friendly curves [8, 9], with a maturing IETF/W3C track for selective disclosure [10], are the concrete choice the desk study made and the one we implement over BLS12-381. Also relevant, and less well known, is SPSEQ-UC [11], structure-preserving signatures on equivalence classes over updatable commitments: it targets re-randomization directly, which is the property a log layer built from these credentials would need and which our evidence-log layer does not yet touch.

Group and ring signatures. Chaum and van Heyst's group signatures [12] and Rivest, Shamir and Tauman's ring signatures [13] are the other route to "some member of a group signed this." Group signatures include an opening authority, which is the shape our escrow layer generalizes to a quorum.

Unlinkable, bounded tokens. Privacy Pass [14] and the VOPRF construction beneath it [15] are the closest deployed analogue to "unlinkable but rate-limited," and the desk study's own cost table lists a Privacy Pass token at roughly 850 µs, sub-ms amortized when pre-fetched in batches — cheap enough that it is plausibly the mechanism that makes an unlinkable design practical rather than merely possible, though it is out of scope for the construction this paper measures.

Threshold cryptography and escrow. Threshold ElGamal, distributed key generation, and Chaum–Pedersen DLEQ proofs for verifiable partial decryption [16] are the machinery that turns "escrow" from a standing backdoor into a quorum action with a public receipt. The literature that keeps that machinery honest is the key-escrow critique in *Keys Under Doormats* [17] and the specific, documented failure modes of the Clipper Chip [18]: a centralized depository, fragile operational control, and no transparency. Section 3 returns to this literature when it explains why our escrow stub's absence from the real half of this tool is not an oversight.

Transport-layer linkability. Oblivious HTTP [19] and relay-based padding schemes are easy to forget and fatal to omit: an unlinkable credential presented over a directly attributable TCP connection is not unlinkable. This construction, and this paper, do not touch the transport layer at all; Section 7 says so explicitly rather than leaving it as a silent gap.

Where this paper sits. None of the above measures what BBS+ costs at agent action rates, composed as a single present-and-verify path, against a fixed real-time budget, on real hardware. That measurement — and the arithmetic error it corrects — is what this paper adds. Nor does any of it ask whether a pre-computation pool holds up under burst, which is the security question the fix for that error raises. In most of this literature pre-computation is a performance detail. Here it is the thing standing between the design and its own latency budget.

3 The Construction We Measured

The desk study's construction has five layers.

Table 1: The five-layer construction P05 proposes. `occultation` measures or models rows 1, 2 and 4 (hardware attestation, agent credential, accountability); the evidence-log and transport rows are out of scope for this paper (Section 7).

Layer	Mechanism	Defeats
Hardware attestation	ECDAA: prove membership in a valid TEE group without emitting a platform identifier	a relying party or auditor fingerprinting the silicon
Agent credential	BBS+ / SPSEQ-UC: fresh blinding per presentation, selective attribute disclosure	colluding relying parties correlating sessions
Evidence log	ephemeral per-session chains, roots posted as re-randomizable Pedersen commitments	a log operator reconstructing a global action sequence
Accountability	k-of-n threshold ElGamal escrow tag with a NIZK that the tag encrypts the credential-bound identity	a rogue escrow node unmasking unilaterally
Transport	OHTTP relay + padding	a network observer linking sessions by IP and timing

Not all five are implemented, and the three we touch are not implemented to the same standard as each other. Table 2 draws the line exactly, and it is required to match `occultation`'s `README.md` member for member. That requirement is deliberate. A reader should be able to check what a paper says it measured against the tool's own documentation, rather than infer it from the fact that Table 1 labelled something a stub — because not every gap between a claim and the code behind it is about a modelled component.

Three of the five rows are what the rest of this paper touches: hardware attestation, the agent credential, and accountability. It is worth being blunt about how little of Section 5 is a timing on this machine. Exactly one table is: Table 4, the agent-credential row, real BLS12-381 pairing arithmetic and a real BBS+ implementation on real hardware. Nothing else in the section is. Table 3's desk-study columns are the published study's own constants, never run here. Table 5

Table 2: What `occultation` is, stated as a three-way split, matching `README.md` exactly. "Real but not interoperable" is not a hedge: the cryptography is genuine and every property below it is asserted by a test, but the wire encoding has not been checked against another implementation.

Category	Members
Real	The Ed25519 baseline, the BLS12-381 group and pairing arithmetic, the pre-computation pool, the anonymity-set calculator, and the timing harness.
Real but not interoperable	BBS+ key generation, signing, verification, presentation and proof verification, implemented over `blstrs` BLS12-381 against `draft-irtf-cfrg-bbs-signatures` [10]. Forged signatures fail, tampered proofs fail, and undisclosed attributes stay undisclosed — each asserted by test. It is not validated against the draft's test vectors, so its wire encoding will not interoperate with another BBS+ implementation.
Modelled — no security whatsoever	ECDAA (`ModelledEcdaa`) and threshold ElGamal escrow (`ModelledThresholdElGamal`). Both carry the correct interface and a representative cost profile and provide no anonymity, no soundness, no confidentiality and no accountability. Neither executes without an explicit `--allow-modelled` flag; asking for one without it exits non-zero and explains why, rather than silently running a component that provides no security.

partitions a fixed TOML file, which has no cryptography in it to time. Section 5.2's burst figures come out of a simulation fed operator-supplied costs. And every figure that touches ECDAA or threshold escrow — the hardware-attestation and accountability rows — is a cost estimate produced by a stub with no cryptographic content in it. We use neither stub to support any claim about security. Only, where unavoidable, about cost, and always labelled.

Leaving escrow out of the real half of Table 2 is a choice, not an oversight, and Section 2's escrow literature is the reason. A k-of-n threshold ElGamal tag with a NIZK of correct encryption is exactly the machinery *Keys Under Doormats* [17] and the Clipper Chip's documented failure [18] warn is easy to build and hard to build safely: the danger sits in the governance around a working implementation as much as in the cryptography. A stub built only to carry a representative cost cannot be mistaken for a governance guarantee we never evaluated. Building a real threshold-opening path and asserting its security is future work. It is not something a cost-measurement tool should do as a side effect.

The evidence-log layer of Table 1 is not implemented at all. It is P05's open questions 1 and 2 (Section 7), and nothing we measured bears on it. The transport layer is not implemented either, but it is a separate omission rather than one of those two questions, and Section 7 names it on its own instead of folding it in.

4 Measuring the Composed Path

The desk study's mistake was structural. It had one budget and two numbers, and it held each number up against the budget on its own. `occultation`'s `bench --composed` exists to make that move unavailable. It folds a presentation cost and a verification cost into one `Composed` value, adds them, and judges the sum — never a half — against the 15.00 ms budget. That budget is written down once, in `src/cost.rs`, and `bench` is the only command that reads it; `pool` and `anonymity`

answer different questions and never consult it. A verdict has three values rather than two: `Fits`, `Misses`, and the one that earns its place, `Marginal`, for a composition under budget by no more than 5 % of it. Drop that third value and 14.90 ms against a 15.00 ms budget prints as a pass — which is the desk study's own verdict on its cached-pairing figure, and the verdict this tool exists to refuse.

`bench --composed` prints two profiles side by side and never merges them. The *desk-study* profile repeats the published figures verbatim, tagged `PUBLISHED`, with the citation attached: constants, not measured on this stack, reported so that anybody can check the composition arithmetic without borrowing our hardware. The *measured* profile times the same operations here, seeded RNG, median of many iterations, tagged `measured here`. They are supposed to disagree in magnitude — they came off different silicon — and Section 5 says by how much and in what shape.

One row of the measured profile carries a third tag, `ESTIMATED`, and what it is not needs saying precisely. Nothing in `occultation` measures a pooled presentation's *online* phase — the work left over once a blinding factor has been pre-computed and pulled off the pool. That row is derived from a cheaper stand-in: two hash-to-scalar operations, standing in for the challenge hash and the field-arithmetic responses a real online phase would do. `occultation pool`, the command that actually drives the pool under load (Section 6), does not close the gap either — when the operator supplies no `--online` cost, it defaults to this same estimate. So the estimate and the pool simulation are not independent evidence for each other. And the row's own note says the estimate is almost certainly too low: it implies a 99 % reduction in presentation cost where the desk study's own pooled figure implies 80 %. We print it anyway, because dropping the row would leave the measured profile's pooled composition unstated altogether, and a caveated estimate beats a silent gap. It is not a claim that we measured a pooled presentation. No number in this paper should be read as one.

5 What It Actually Costs

Every table and every inline figure in this section is generated by `scripts/regen-results.sh` from files under `../results/` and either `\input`-ed or substituted as a LaTeX macro. None of it was typed. Files under `results/deterministic/` are arithmetic over the desk study's published constants, a seeded pool simulation (`pool-facts.tex` included), or a partition of a fixed fleet file, so they come out byte-for-byte identical on any host and continuous integration fails the build on a diff. Files under `results/measured/` are wall-clock timing. They differ by host, so they are not diffed, and every number taken from them is marked as measured — in its own table, or, for `composed-facts.tex`, in the prose that quotes it.

5.1 Adding the two halves together

Table 3 is the desk study's own published figures, composed as one path instead of two. The naive composition misses the budget outright. Caching pairings pulls it back to `MARGINAL` rather than a pass — under the limit, but by less than the jitter of an uncontended context switch. Only the pre-computation pool, together with cached pairings, fits with room. That is the headline correction, and it is a statement about these figures: in the desk study's own numbers, pre-computation is not an optimization sitting on top of a design that already works. It is the reason the design fits its budget at all.

Table 4 times the individual primitives underneath Table 3's presentation and verification costs, here on this host rather than read out of a publication: ten primitive operations instead of three

Composition	Present	Verify	Total	Verdict
naive (no pool, no cached pairings)	13.80 ms	4.20 ms	18.00 ms	MISSES by 3.00 ms
cached pairings only	13.80 ms	1.10 ms	14.90 ms	MARGINAL — 0.10 ms spare
pre-computation pool + cached pairings	2.80 ms	1.10 ms	3.90 ms	fits, 11.10 ms spare

Table 3: The composed present-and-verify path against the 15.00 ms budget: the desk study's published figures for presentation and verification, assessed together rather than separately as the desk study assessed them. The naive composition sums to 18.00 ms and misses the budget by 3.00 ms even though each half alone appeared to fit. Caching pairings brings the sum to 14.90 ms, 0.10 ms under the limit — not headroom, hence `MARGINAL` rather than a pass. Only the pre-computation pool, combined with cached pairings, fits with room to spare: 3.90 ms total, 11.10 ms of headroom. `results/measured/composed.txt` reports the profile measured on this host, which diverges from these published figures by roughly an order of magnitude without changing the correction, which is an error in arithmetic, not in hardware.

compositions, and no budget verdict against any one of them, which is the whole point Table 3 exists to make.

Operation	Cost	P95	×Ed25519	Provenance
Ed25519 sign	0.007 ms	0.007 ms	0.3	measured here (n=200)
Ed25519 verify	0.018 ms	0.019 ms	0.7	measured here (n=200)
Ed25519 sign + verify (baseline)	0.025 ms	–	1.0	measured here (n=200)
G1 scalar multiplication	0.039 ms	0.040 ms	1.6	measured here (n=200)
hash_to_scalar	0.002 ms	0.002 ms	0.1	measured here (n=200)
BBS+ sign (issuer)	0.208 ms	0.219 ms	8.2	measured here (n=200)
BBS+ signature verify (holder)	0.642 ms	0.698 ms	25.3	measured here (n=200)
BBS+ presentation	0.539 ms	0.564 ms	21.2	measured here (n=200)
BBS+ proof verify	0.766 ms	0.793 ms	30.2	measured here (n=200)
BBS+ proof verify, cached pairings	0.684 ms	0.713 ms	27.0	measured here (n=200)

Table 4: Primitive costs measured on this host against an Ed25519 sign-and-verify round trip. ECDAA and threshold escrow are omitted: they are modelled stubs and this table is generated from a plain `occultation bench` run, which never executes a modelled component. See `results/measured/HEADER.txt` for the exact host and toolchain.

The measured composed path (`results/measured/composed.txt`, given in prose here rather than as a table) is 14× cheaper here than the desk study's published figures imply, and the composed BBS+ path costs 51× an Ed25519 sign-and-verify round trip on this host. That factor is not like-for-like. The desk study's naive presentation figure folds in a 4.6 ms (33 %) allowance for the escrow tag; the measured figure here leaves escrow out entirely. Take escrow off both sides and the divergence is roughly 10×. None of which touches the composition error, because that error is arithmetic rather than a property of any one machine, and it comes out the same wherever the desk study's published figures are added up.

The magnitude is the part that hides the finding. The divergence is not uniform. Cached pairings are the one optimization measured on both sides: the desk study's figures credit caching the pairing computation with 74 % of verification cost, and the same optimization saves 11 % here. A host that were simply faster across the board would keep the desk study's ratio between optimized and unoptimized verification. This host does not. So the measured block does not corroborate the

desk study's optimization structure, and we report that as a finding in its own right rather than averaging it away as noise.

5.2 What the pool costs when it runs dry

Section 6 argues that what the correction above really raises is the pool's soundness, not its speed. These are the numbers behind that argument. They come from the shipped burst configuration — capacity 64, refill 700 µs, online 60 µs, burst rate 5000 s^{-1}, seed 7, the same configuration `tests/acceptance.rs`'s acceptance test 2 exercises. Of 10 000 requests over a 2.0 s window, 89 were served immediately and 9911 stalled. The worst response was 82 591× an idle one.

That last figure is a peak over the observation window, and it grows linearly with the window, so it should never travel on its own. Beside it go the three numbers that do not depend on how long you watch: an overload factor of 3.50×, arrivals over refill; a growth rate of 8.33× an idle response, added to each successive stall once the pool is dry; and an onset at request 89, which is how briefly a burst has to be sustained before the channel opens. Every one of 10 000 requests got a distinct blinding factor. Reuse events: 0.

The refill and online costs above (700 µs and 60 µs) are values `scripts/regen-results.sh` stipulates for this simulation. They are not a measurement of any deployment. Both 82 591× and 8.33× scale inversely with whatever online cost you pick; 3.50× does not depend on it at all.

5.3 How much of the fleet looks alike

Size	Host group	PCE SVN	Microcode	TDX module
71	us-east-pool-patched	13	0x2b000603	1.5.05
34	us-east-pool-pending	12	0x2b000603	1.5.05
11	eu-west-rack-7	12	0x2b000401	1.5.03
3	us-east-canary	13	0x2b000701	1.5.05
1	us-east-rebuilt-0419	14	0x2b000701	1.5.06

Table 5: Anonymity sets for fleet `acme-prod-drifted`: 120 hosts of identical hardware, after six weeks of ordinary operational drift in patch level, microcode and PCE SVN. 5 distinct TCB configurations emerge; the largest anonymity set is 71 of 120 hosts, and 1 host(s) are alone on their configuration — no credential scheme can help them, because the quote collateral identifies the machine before any credential is even presented. Effective set size 52.73 of 120; entropy 1.470 bits.

Table 5 answers a question the desk study's cost analysis never asks. Suppose the credential layer is perfectly unlinkable. How large is the anonymity set the hardware attestation layer leaves standing underneath it? `examples/fleet-drifted.toml` is a synthetic, hand-authored fixture, not telemetry from any live fleet. It describes 120 hosts of identical hardware after six weeks of the kind of operational drift a real fleet accumulates: a canary batch here, a delayed patch there, a microcode revision that reached some racks and not others. A uniform fleet with none of that drift (`examples/fleet-uniform.toml`) is one anonymity set the size of the whole fleet, by construction, and exists only as the tool's control case. Table 5 is the drifted fleet, which is closer to how a fleet actually runs — though both files are things we wrote, not telemetry from a live fleet.

6 Is the Pre-Computation Pool Sound?

Section 5.1 established that in the desk study's own published figures, pre-computation is what lets the composed path fit its budget at all, not an optional speed-up on top of a design that already worked. That moves the risk somewhere else. The question stops being whether BBS+ is fast enough and becomes whether the pool that makes it fast enough is sound. Nobody asked before, because until the arithmetic error surfaced, pre-computation looked like an optimization, and optimizations are not where a security review looks.

The two available behaviours. A pool of pre-computed blinding factors, sitting in front of a burst of demand, has exactly two things it can do once it is empty. It can make the caller wait for a fresh item, or it can hand back one that has already been used. `occultation`'s `PrecomputationPool` does the first and never the second. On exhaustion `acquire` returns `None` rather than a used item, and there is no flag and no code path that makes it return one. Section 5.2 is the price of that choice: under a sustained burst the delay does not plateau, it grows with the length of the burst, and that growth is a timing side channel with a duration-invariant signature. Watch nothing but an agent's response latency and you can read off how overloaded it is and how long it has been that way.

Why the alternative is not a lesser evil. Reusing one set of blinding scalars across two presentations does not merely weaken unlinkability. It discloses the credential outright, and that is provable rather than asserted. A BBS+ presentation's proof of knowledge has the Schnorr-style shape common to sigma protocols: for a hidden value x (the signature's blinding scalar, or an undisclosed attribute) with blinding $\tilde{x}$ and Fiat–Shamir challenge c, the response the verifier sees is

$$\hat{x} = \tilde{x} + c \cdot x \pmod{r}. \tag{1}$$

If two presentations to two different relying parties reuse the same blinding $\tilde{x}$ but necessarily bind to two different challenges $c_1 \neq c_2$ — the challenge is derived from a per-session nonce, so identical challenges would themselves require a coincidence — then subtracting the two responses cancels the blinding entirely:

$$\hat{x}_1 - \hat{x}_2 = (c_1 - c_2)\, x \quad \Longrightarrow \quad x = \frac{\hat{x}_1 - \hat{x}_2}{c_1 - c_2}. \tag{2}$$

This is not a hypothetical weakness. `pool.rs`'s test `reusing_one_set_of_scalars_leaks_the_signature_and_every_hidden_attribute` does exactly this division on the reused presentation's responses, against a real signature and real hidden attributes, and asserts that what comes back is the signature's scalar and every undisclosed attribute in order. The two presentations are linkable on sight before any of that arithmetic is done anyway, because they carry the same commitment values. Reuse is not a milder version of unlinkability failing. It is unlinkability's negation, in closed form, handed to anyone who sees both transcripts.

What the type system buys, stated precisely. `PoolItem` is not `Clone`, and its `consume` method takes `self` by value, so *accidental* reuse — holding an item, copying it, or pulling it off the pool a second time — is impossible. The fields are private, the type cannot be duplicated, and `acquire` never puts an item back in its own queue. This is worth stating exactly, because it is easy to overstate. *Deliberate* reuse is not prevented. `consume` puts no bound on what its closure returns, the scalars underneath are an ordinary `Clone`able type with public fields, so `item.consume(|s| s.clone())` yields an owned copy that outlives the item — and the lower-level proving function

that takes pre-chosen scalars is public too. What the design buys is not a guarantee that reuse cannot happen. It is that any reuse is one conspicuous, greppable `.clone()` at the point it happens, rather than something a pool running dry under load produces on its own — which is exactly the failure mode this paper's measurement is about. A reviewer looking for that call will find it. A load test will not, because nothing under load produces it. Exhaustion produces a stall, and only a stall.

What we are not claiming. We have not built or evaluated any third option beyond stall and reuse. P05's own open question — whether pool refill, running as idle-time work inside the protected environment, correlates with agent activity closely enough that an observable host resource (power draw, cache behaviour, scheduling) gives it away — is untouched by anything we measured. And `pool`'s simulation treats refill as a fluid approximation of a single worker rather than a discrete-event queue. The approximation is written down in the code and it does not change which of the two behaviours gets chosen, but it does mean the specific latency numbers in Section 5.2 are a model of one refill strategy rather than a measurement of every plausible one.

7 What We Still Don't Know

Four questions this tool does not answer, in the order the desk study's own open-questions list puts them:

1. **Can a log operator correlate re-randomized commitments by posting time and size, even if the commitments themselves hide everything cryptographically?** The construction's evidence-log layer proposes per-session ephemeral chains with roots posted as re-randomizable Pedersen commitments, specifically so that a log operator cannot reconstruct a global action sequence out of what the commitments contain. Nothing in this tool touches that layer — it is not implemented, measured, or modelled here. Hiding a value cryptographically says nothing about when and how often the value gets posted, and an operator who can watch posting times and sizes has a side channel no property of the commitment scheme addresses. This is the gap P05 names, and the thing to prove or refute. This paper does neither.

2. **What does tamper-evidence mean once the global chain is replaced by independent per-session chains?** A single global hash chain proves that no action was deleted. Independent per-session chains, the construction's own proposal for unlinkability, prove only that no action was deleted *from a session an auditor already knows exists*: an agent that suppresses an entire session leaves no gap for anyone to notice. This may be the more consequential result available from this whole research direction — that unlinkability and completeness-of-record are in tension rather than independently achievable — and this paper's measurements say nothing about it either way, because the evidence-log layer is out of scope here.

3. **Revocation.** Revoking a compromised agent's credential without linking its past actions, and without an accumulator update cost that scales badly with fleet size, is historically the hardest part of deploying anonymous credentials, and it is barely touched by the desk study this paper corrects. `occultation` has no revocation mechanism, measured or modelled.

4. **Is an anonymity set built out of TCB configuration diversity even the right set to be measuring?** Section 5.3 answers "how large is the set a fleet's hardware diversity

leaves standing," which is worth answering on its own. It is not the same question as "what anonymity set should a colluding pair of relying parties be evaluated against." The right set might be agents inside one enterprise, or enterprises sharing one credential issuer, or TEE hardware generations across many unrelated fleets — and the answer changes who the scheme protects from whom. Our calculator partitions whatever fleet description you hand it. It has no opinion about which fleet is the relevant one. Supplying that opinion is future work, not something this measurement settles on its own.

Two structural omissions sit above all four and are worth naming out loud. The evidence-log and transport layers of Table 1 are not implemented, measured, or modelled anywhere in this artifact. The first is questions 1 and 2 above. The second — Oblivious HTTP, relay padding, and everything else that stops an unlinkable credential from riding over a directly attributable connection — is named by P05 and untouched by this tool entirely. Every figure this paper reports, about cost or exhaustion or anonymity, is a statement about the credential and hardware-attestation layers alone.

8 Conclusion

Two numbers were held up against one budget, one at a time, and each of them cleared it. The design they belong to was reported as fitting. It does not fit: the sum is over. That is the correction, it needs no hardware to reproduce, and it comes out the same wherever the desk study's published figures are added up. What it changes is what the design is standing on. In those figures the pre-computation pool is not a speed-up bolted onto something that already worked — it is the reason anything fits, which turns a performance detail into a security question nobody had reason to ask. So we asked it. The pool stalls rather than reusing, because reuse hands anybody who sees two transcripts the signature scalar and every undisclosed attribute in closed form; and stalling is not free either, because the delay grows with the burst and an observer can read the overload off an agent's response latency.

The part of this we are least able to defend is the evidence underneath it. The fleet whose anonymity sets we partition is a TOML file we wrote by hand, and the drift in it is the drift we thought a real fleet would accumulate, not drift anybody observed. Two of the five layers are stubs with no security in them whatsoever. The BBS+ under the credential layer is real cryptography, but it has never been run against the IETF draft's test vectors and will not interoperate with another implementation. An error in somebody else's arithmetic is cheap to be right about, because the arithmetic is public and anyone can redo it. Everything else here rests on fixtures we wrote for ourselves. If one sentence survives this paper, we would rather it be that one than the correction.

References

[1] Task Force for AI Agents in Healthcare. occultation: measuring what verifiable unlinkability costs at agent action rates. `https://github.com/Task-force-for-AI-agents-in-Healthcare/occultation`, 2026. Apache-2.0.

[2] Task Force for AI Agents in Healthcare. Open issues, appendix d — proof-of-control standard. `https://github.com/Task-force-for-AI-agents-in-Healthcare/ov-poc-standard/blob/master/0.1/en/0x93-Appendix-D_Open-Issues.md`, 2026. Open issue 2, on verifiable unlinkability.

[3] Ernie Brickell, Jan Camenisch, and Liqun Chen. Direct anonymous attestation. In *Proceedings of the 11th ACM Conference on Computer and Communications Security (CCS)*, pages 132–145, 2004.

[4] International Organization for Standardization. Iso/iec 20008-2:2013 — information technology — security techniques — anonymous digital signatures — part 2: Mechanisms using a group public key, 2013.

[5] Jan Camenisch and Anna Lysyanskaya. Efficient non-transferable anonymous multi-show credentials with optional anonymity revocation. In *Advances in Cryptology — EUROCRYPT 2001*, pages 93–118, 2001.

[6] Jan Camenisch and Els Van Herreweghen. Design and implementation of the idemix anonymous credential system. In *Proceedings of the 9th ACM Conference on Computer and Communications Security (CCS)*, 2002. [verify] exact page range against the primary proceedings.

[7] Christian Paquin and Greg Zaverucha. U-prove cryptographic specification v1.1 (revision 3). Technical report, Microsoft Corporation, 2013. [verify] revision number against the current published specification.

[8] Dan Boneh, Xavier Boyen, and Hovav Shacham. Short group signatures. In *Advances in Cryptology — CRYPTO 2004*, pages 41–55, 2004. The BBS signature this paper's "BBS+" variant, as specified in [10], extends; the "+" extension is commonly attributed to Au, Susilo and Mu (2006) [verify exact citation].

[9] Yumi Sakemi, Tetsutaro Kobayashi, Tsunekazu Saito, and Riad S. Wahby. Pairing-friendly curves. IRTF Internet-Draft, draft-irtf-cfrg-pairing-friendly-curves, 2022. Defines the BLS12-381 curve used by this paper's implementation, via the `blstrs` crate.

[10] Tobias Looker, Sean Bowe, Vasilis Hoyt, Manu Sporny, and Sam Bachhuber. The bbs signature scheme. IRTF Internet-Draft, draft-irtf-cfrg-bbs-signatures, 2026. [verify] author list and draft revision against the current IETF datatracker entry.

[11] Georg Fuchsbauer, Christian Hanser, and Daniel Slamanig. Structure-preserving signatures on equivalence classes and constant-size anonymous credentials. *Journal of Cryptology*, 2019. [verify] the specific "SPSEQ-UC" updatable-commitment variant P05 names against its primary source.

[12] David Chaum and Eugène van Heyst. Group signatures. In *Advances in Cryptology — EUROCRYPT '91*, pages 257–265, 1991.

[13] Ronald L. Rivest, Adi Shamir, and Yael Tauman. How to leak a secret. In *Advances in Cryptology — ASIACRYPT 2001*, pages 552–565, 2001.

[14] Alex Davidson, Jana Iyengar, and Christopher A. Wood. The privacy pass protocol. RFC 9578, 2024.

[15] Alex Davidson, Armando Faz-Hernandez, Nick Sullivan, and Christopher A. Wood. Oblivious pseudorandom functions (oprfs) using prime-order groups. RFC 9497, 2023.

[16] David Chaum and Torben P. Pedersen. Wallet databases with observers. In *Advances in Cryptology — CRYPTO '92*, pages 89–105, 1992.

[17] Harold Abelson, Ross Anderson, Steven M. Bellovin, Josh Benaloh, Matt Blaze, Whitfield Diffie, John Gilmore, Matthew Green, Susan Landau, Peter G. Neumann, Ronald L. Rivest, Jeffrey I. Schiller, Bruce Schneier, Michael A. Specter, and Daniel J. Weitzner. Keys under doormats: Mandating insecurity by requiring government access to all data and communications. Technical Report MIT-CSAIL-TR-2015-026, MIT CSAIL Technical Report, 2015.

[18] Matt Blaze. Protocol failure in the escrowed encryption standard. *Proceedings of the 2nd ACM Conference on Computer and Communications Security (CCS)*, pages 59–67, 1994.

[19] Martin Thomson and Christopher A. Wood. Oblivious http. RFC 9458, 2023.